

Análise Assintótica

Henrique do Nascimento Cunha, MSc.

Análise de algoritmos

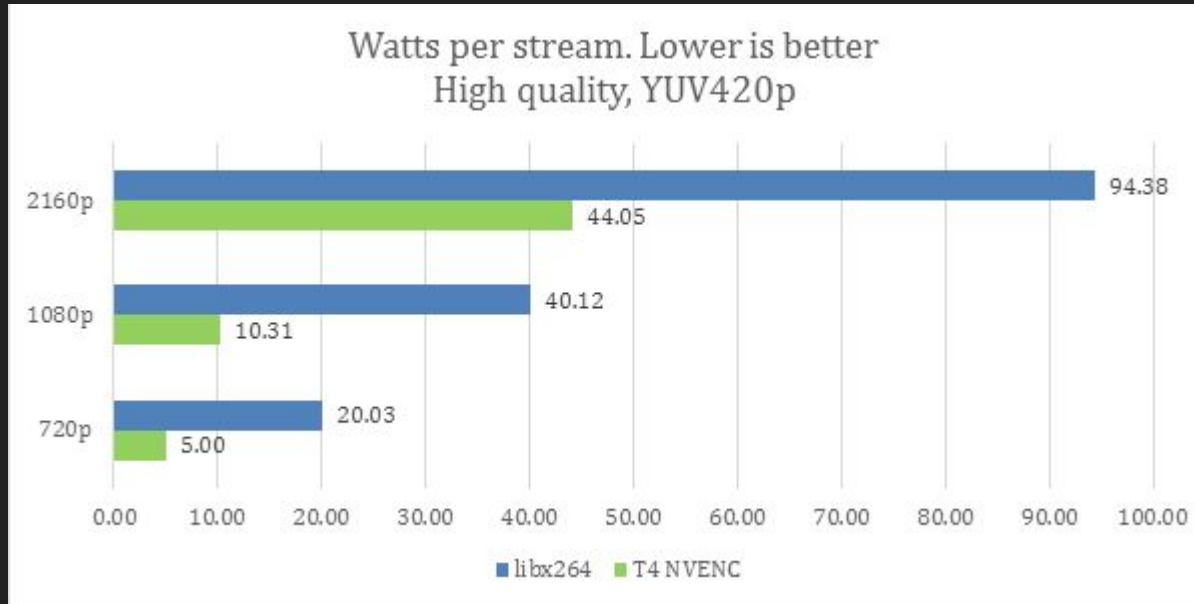
- O que é um algoritmo eficiente?
 - Processamento
 - Memória
 - Largura de banda
 - Complexidade em termos assintóticos
- O que é um algoritmo escalável?
- Como avaliar eficiência
 - Experimentos (*benchmarks*)
 - Análise de algoritmos

Análise de algoritmos

- Avaliação experimental
 - Identificar gargalos
 - verificar desempenho
 - Adequação a requisitos de tempo real
 - Limitações:
 - Especificidades de *hardware*
 - Escrever e testar o algoritmo
 - Limitação de casos de teste ou cenários

Análise de algoritmos

- Avaliação experimental
 - Exemplo: <https://developer.nvidia.com/blog/turing-h264-video-encoding-speed-and-quality/>



Análise de algoritmos

- Análise assintótica
 - Permite **prever** os recursos necessários para execução
 - Análise de **viabilidade**
 - **Comparação** com outros algoritmos
 - Independente de *hardware*

Análise de algoritmos

- Como medir eficiência:
 - Avaliação de diversos aspectos
 - Uso de memória
 - Tempo de execução
 - Consumo de energia
 - Expressar o tempo de execução de um algoritmo como função do tamanho de sua entrada
 - Quantidade de elementos de um array
 - Vértices ou arestas de um grafo

Análise de algoritmos

SubtraiExtremos(v)

1 $n = \text{len}(v)$

2 **return** $v[n-1] - v[0]$ # Custo k

Complexidade: $O(k)$, sendo k um valor constante

Análise de algoritmos

Max(v)

```
1  n = len(v)
2  maior = v[0]
3  for i in range(n):
4      if (v[i] > maior):
5          maior = v[i]
6  return maior
```

Complexidade: $O(n)$

Análise de algoritmos

ContaElemComum(v1, v2)

1	n = len(v1)	# Custo 1
2	qtd = 0	# Custo 1
3	for i in range(n):	# Custo n
4	for j in range(n):	# Custo n^2
5	if (v1[i] == v2[j]):	# Custo n^2
6	qtd += 1	# Custo n^2
7	return maior	# Custo 1

Complexidade: $O(n^2)$

Análise de algoritmos

```
1  from math import fabs
2  n = int(input())
3  v = []
4
5  for i in range(n):
6      val = int(input())
7      v.append(fabs(val))
8
9  for i in range(n):
10     ma = max(v)
11     print(ma)
12     v[v.index(ma)] = -1
```

Análise de algoritmos

```
1  from math import fabs
2  n = int(input())
3  v = []
4
5  for i in range(n):
6      |   val = int(input())
7      |   v.append(fabs(val))
8
9  for i in range(n):
10     |   ma = max(v)
11     |   print(ma)
12     |   v[v.index(ma)] = -1
```

Complexidade: $O(N^2)$

Análise de algoritmos

Tamanho máximo de n	Complexidade (pior caso)	Descrição
$\leq 100M$	$O(1)$	- Somar 2 elementos de um vetor
$\leq 100M$	$O(\log(n))$	- Busca binária - Inserção ordenada em árvore binária de busca
$\leq 100M$	$O(n)$	- Um for de 1 a n
$\leq 1M$	$O(n\log(n))$	- Ordenação (quick sort, merge sort)
$\leq 10K$	$O(n^2)$	- Dois comandos for (um dentro do outro) de 1 a n - Ordenação (bubble sort, insertion sort)

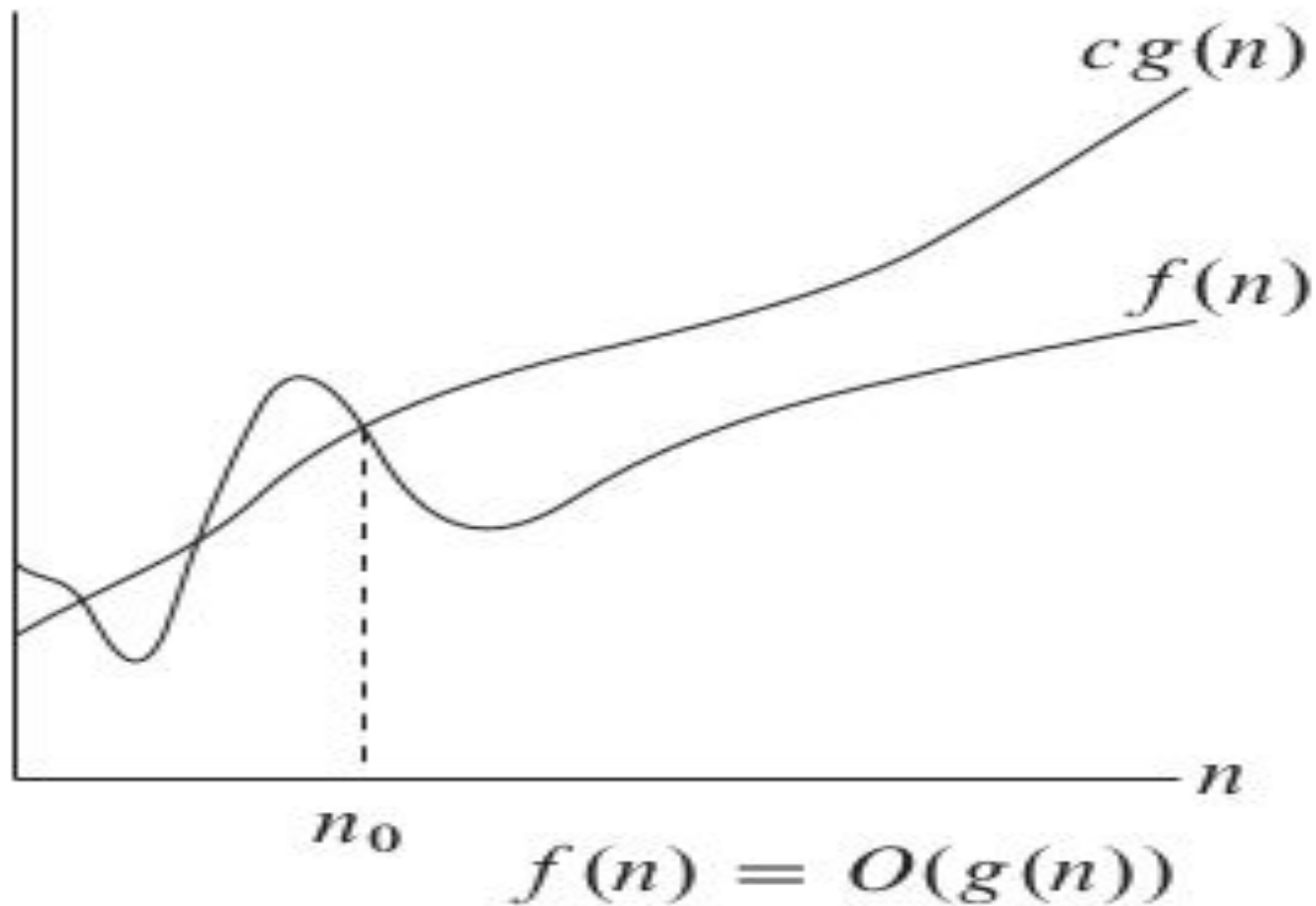
Análise assintótica

- Caracterização do tempo de execução em situação limite
- Algoritmos assintoticamente mais eficientes são melhores para entradas grandes (nem sempre para entradas pequenas)
- “Ignoramos” os tempos específicos de cada operação quando esses tempos são constantes

Análise assintótica

- Notação Big-O
- Dado um algoritmo com tempo de execução $f(n)$, $f(n) = O(g(n))$, se existir uma constante c e um valor inicial n_0 , tal que:
 - $f(n) \leq cg(n), \forall n \geq n_0$

Análisis



Análise assintótica

- Exemplo:
 - Mostre que:

$$n^2/2 - 3n = O(n^2)$$

Análise assintótica

Tabela de complexidade assintótica

$O(1)$
$O(\log n)$
$O(n)$
$O(n \log n)$
$O(n^2)$
$O(n^k)$, para k constante e maior que 2
$O(2^n)$
$O(n!)$
$O(2^{n!})$

Menos complexo

Mais complexo

Análise assintótica

- Em uma função de crescimento o que importa é o **termo de maior crescimento assintótico**
- Exemplos:
 - $4n + n^2 + 75$
 - $n \log_2 n + 12n^2 - 20$
 - $n^{100} + 2^n$
 - $n + 175 \log n$

Análise assintótica

- Em uma função de crescimento o que importa é o **termo de maior crescimento assintótico**
- Exemplos:
 - $4n + n^2 + 75 \rightarrow$ Termo de maior crescimento: n^2
 - $n \log_2 n + 12n^2 - 20 \rightarrow$ Termo de maior crescimento: n^2
 - $n^{100} + 2^n \rightarrow$ Termo de maior crescimento: 2^n
 - $n + 175 \log n \rightarrow$ Termo de maior crescimento: n

Análise assintótica

- **Constantes** que estejam multiplicando termos da equação não interessam!
- Exemplo:
 - $O(2n) = O(n)$
 - $O(20n^2) = O(n^2)$
 - $n^4 + n^3 + n^2 + n = O(n^4)$
 - $O(42n + 0,5n^3 + 314) = O(n^3)$
 - $O(4n\log n + 2n) = O(n\log n)$